MUSTAFACODE

Lecture 22

Inheritance in C++ (Detailed Notes with Real Examples)
Object-Oriented Programming (CS304)

Course Code: **CS304**

Publisher: **mustafacode**

Compiled On: **May 29, 2026**

CS304 OBJECT-ORIENTED PROGRAMMING

Lecture 22: Inheritance in C++ (Detailed Notes with Real Examples)

Lecture No.22 — Inheritance in C++ (Detailed Notes with Real Examples)

🔷 1. Inheritance Introduction

Inheritance OOP ka ek important concept hai jisme ek class (Child/Derived) doosri class (Parent/Base) ki properties aur behavior inherit karti hai.

👉 Simple meaning:
Ek class doosri class ka **reusable version** ban jati hai.

📌 Real Life Example:

- Person → Base Class
- Student → Derived Class

👉 Student IS A Person
👉 Teacher IS A Person

Is se pata chalta hai ke common properties reuse ho rahi hoti hain.

🔷 2. Base Class & Derived Class

✓ Base Class (Parent)

Wo class jiska data aur functions inherit kiye jate hain.

✓ Derived Class (Child)

Wo class jo base class ko extend karti hai aur extra features add karti hai.

📌 Example:

```
class Person {
public:
    string name;
    int age;

    void displayPerson() {
        cout << name << " " << age << endl;
    }
};

class Student : public Person {
public:
    int rollNo;

    void displayStudent() {
        cout << rollNo << endl;
    }
};
```

🔷 3. Types of Inheritance in C++

C++ me inheritance 3 types ki hoti hai:

- Public Inheritance
- Private Inheritance
- Protected Inheritance

👉 Most common: Public Inheritance

📌 Example:

```
class Student : public Person {
};
```

🔷 4. IS-A Relationship

Inheritance IS-A relationship show karti hai.

📌 Real Examples:

- Car IS A Vehicle
- Dog IS A Animal
- Student IS A Person

👉 Matlab derived class base class ka specialized form hoti hai.

🔷 5. UML Representation

UML diagram me inheritance arrow se show hota hai:

```
Student  --------> Person
```

👉 Arrow child se parent ki taraf hota hai.

🔷 6. Member Access Rules

✓ Public Members:

Derived class me accessible hote hain.

❌ Private Members:

Direct access nahi hota.

👉 Encapsulation ki wajah se private members hidden rehte hain.

📌 Example:

```
class Person {
private:
    string name;

public:
    string getName() {
        return name;
    }
};

class Student : public Person {
public:
    void show() {
        // cout << name; ❌ ERROR
        cout << getName(); // ✓ Correct
    }
};
```

🔷 7. Memory Representation

Derived class object memory me aise store hota hai:

👉 Base class ka part + Derived class ka part

📌 Example:

```
Student object:

[ Person Data ]
   name
   age

[ Student Data ]
   rollNo
   semester
```

🔷 8. Constructors in Inheritance

✓ Rule:

- Pehle base class constructor run hota hai
- Phir derived class constructor run hota hai

📌 Example:

```
class Parent {
public:
    Parent() {
        cout << "Parent Constructor\n";
    }
};

class Child : public Parent {
public:
    Child() {
        cout << "Child Constructor\n";
    }
};

int main() {
    Child c;
}
```

📌 Output:

```
Parent Constructor
Child Constructor
```

🔷 9. Constructor Problem (No Default Constructor)

Agar base class me default constructor nahi ho to error aata hai.

❌ Example:

```
class Parent {
public:
    Parent(int x) {}
};

class Child : public Parent {
public:
    Child() {}
}; // ERROR
```

✓ Solution: Base Class Initializer

```
class Parent {
public:
    Parent(int x) {}
};

class Child : public Parent {
public:
    Child(int x) : Parent(x) {
    }
};
```

🔷 10. Base Class Initializer

Derived class base class constructor ko explicitly call karti hai.

📌 Example:

```
class Parent {
public:
    Parent(int x) {
        cout << "Parent Constructor\n";
    }
};

class Child : public Parent {
public:
    Child(int x) : Parent(x) {
        cout << "Child Constructor\n";
    }
};
```

🔷 11. Member Initialization

Derived class apne members ko initialize kar sakti hai, lekin base class members ko direct initialize nahi kar sakti.

❌ Wrong:

```
class Student : public Person {
public:
    Student(int a) : age(a) {} // ERROR
};
```

✓ Correct:

```
class Student : public Person {
public:
    Student(int a) {
        // use setter or constructor
    }
};
```

🔷 12. Destructors in Inheritance

Destructors reverse order me call hote hain:

👉 Pehle derived class destructor
👉 Phir base class destructor

📌 Example:

```
class Parent {
public:
    Parent() {
        cout << "Parent Constructor\n";
    }

    ~Parent() {
        cout << "Parent Destructor\n";
    }
};

class Child : public Parent {
public:
    Child() {
        cout << "Child Constructor\n";
    }

    ~Child() {
        cout << "Child Destructor\n";
    }
};

int main() {
    Child c;
}
```

📌 Output:

```
Parent Constructor
Child Constructor
Child Destructor
Parent Destructor
```

🔷 13. Important Rules Summary

✓ Base class constructor pehle execute hota hai
✓ Derived class constructor baad me execute hota hai
✓ Destructors reverse order me execute hote hain
✓ Private members inherit hote hain but access nahi hote
✓ Base class initializer se constructor call hota hai

🔷 14. Real World Example (Complete)

```
class Person {
public:
    string name;

    Person(string n) {
        name = n;
    }

    void showPerson() {
        cout << name << endl;
    }
};

class Student : public Person {
public:
    int rollNo;

    Student(string n, int r) : Person(n) {
        rollNo = r;
    }

    void showStudent() {
        cout << name << " " << rollNo << endl;
    }
};

int main() {
    Student s("Tehreem", 101);
    s.showStudent();
}
```

📌 Output:

```
Tehreem 101
```

🎯 Final Summary

Inheritance ka main goal:

✓ Code reuse
✓ Real-world modeling
✓ Cleaner design
✓ Reduce duplication
